certificateAuthorityManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"certificateAuthorityManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `certificateAuthorityManagement` service of the CertificateAuthority system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation list-certificates . . . 4
2.2 operation revoke-certificate . . . 4
2.3 operation revoke-certificate-by-content . . . 5
2.4 operation list-keys . . . 5
2.5 operation add-key . . . 6
2.6 operation delete-key . . . 6

3 Data Models **7**
3.1 struct CertificateRecord . . . 7
3.2 struct RemovalResult . . . 7
3.3 struct RevocationRequest . . . 7
3.4 struct RevocationResult . . . 7
3.5 struct TrustedKeyRecord . . . 7
3.6 struct TrustedKeyRequest . . . 8
3.7 struct TrustedKeyResult . . . 8
3.8 Primitives . . . 8

4 References **9**

5 Revision History **10**
5.1 Amendments . . . 10
5.2 Quality Assurance . . . 10

1 Overview

This document describes the `certificateAuthorityManagement` service interface of the CertificateAuthority system, which provides operator management of the certificate register and of trusted keys, including revocation.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `certificateAuthorityManagement` service interface

This document provides the Interface Design Description IDD to the *certificateAuthorityManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `certificateAuthorityManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https`, policy `CERT_AUTH`, access address `certificate-authority`, port 8448 and base path `/certificate-authority`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`list-certificates`	GET	`/mgmt/certificates`
`revoke-certificate`	DELETE	`/mgmt/certificates/{id}`
`revoke-certificate-by-content`	POST	`/mgmt/revocations`
`list-keys`	GET	`/mgmt/keys`
`add-key`	PUT	`/mgmt/keys`
`delete-key`	DELETE	`/mgmt/keys/{id}`

2 Service Operations

The interfaces of the `certificateAuthorityManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `certificateAuthorityManagement` SD document, as well as input and output types.

Every request needs mutual TLS with a system or operator certificate of the cloud and `Authorization: Bearer IDENTITY-TOKEN//<token>`; the token is verified with Authentication and must belong to the system named by the certificate. Request and response bodies are `application/json` unless stated otherwise. Errors are answered with `{"error": "<text>"}` and the status codes `400 Bad Request` (invalid input), `401 Unauthorized` (no valid identity), `403 Forbidden` (caller not permitted), `404 Not Found`, `500 Internal Server Error`, `502 Bad Gateway` (a system it depends on failed) and `503 Service Unavailable` (overloaded).

2.1 GET /certificate-authority/mgmt/certificates

Operation: list-certificates
Output: Array<CertificateRecord>

Returns the certificate register, newest first.

A successful call returns `200 OK` with the output in the body.

```
GET /certificate-authority/mgmt/certificates HTTP/1.1
Host: certificate-authority:8448
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 1: An list-certificates invocation.

```
[
  {
    "id": 42,
    "serial_number": "4189327123456789",
    "common_name": "Sensor1.TestCloud.Company.arrowhead.eu",
    "created_by": "Onboarding",
    "valid_from": "2026-10-22T12:00:00Z",
    "valid_until": "2027-10-22T12:00:00Z",
    "certificate_pem": "-----BEGIN CERTIFICATE-----\\n..."
  }
]
```

Listing 2: An list-certificates response.

2.2 DELETE /certificate-authority/mgmt/certificates/*{id}*

Operation: revoke-certificate
Input: String
Output: RemovalResult

Revokes a register record by identifier (`{id}`) and republishes the CRL.

Path parameter: *id*.

A successful call returns `200 OK` with the output in the body.

```
DELETE /certificate-authority/mgmt/certificates/<id> HTTP/1.1
Host: certificate-authority:8448
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An revoke-certificate invocation.

```
{
  "success": true
}
```

Listing 4: An revoke-certificate response.

2.3 POST /certificate-authority/mgmt/revocations

Operation: revoke-certificate-by-content
Input: RevocationRequest
Output: RevocationResult

Revokes any certificate of this CA given by content; unknown ones are added to the register as revoked.
A successful call returns `200 OK` with the output in the body.

```
POST /certificate-authority/mgmt/revocations HTTP/1.1
Host: certificate-authority:8448
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "certificate": "<PEM or Base64 DER>"
}
```

Listing 5: An revoke-certificate-by-content invocation.

```
{
  "id": 42,
  "serialNumber": "4189327123456789",
  "revoked": true,
  "changed": true
}
```

Listing 6: An revoke-certificate-by-content response.

2.4 GET /certificate-authority/mgmt/keys

Operation: list-keys
Output: Array<TrustedKeyRecord>

Returns the trusted keys.
A successful call returns `200 OK` with the output in the body.

```
GET /certificate-authority/mgmt/keys HTTP/1.1
Host: certificate-authority:8448
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 7: An list-keys invocation.

```
[
  {
    "id": 42,
    "public_key_pem": "-----BEGIN PUBLIC KEY-----\\n...",
    "description": "Display may read",
    "valid_after": "string",
    "valid_before": "string",
    "created_at": "string"
  }
```

```
]
```

Listing 8: An list-keys response.

2.5 PUT /certificate-authority/mgmt/keys

Operation: add-key
Input: TrustedKeyRequest
Output: TrustedKeyResult

Adds a trusted key with validity.
A successful call returns `201 Created` with the output in the body.

```
PUT /certificate-authority/mgmt/keys HTTP/1.1
Host: certificate-authority:8448
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "publicKey": "-----BEGIN PUBLIC KEY-----\\n...",
  "description": "Display may read",
  "validAfter": "2026-10-22T12:00:00Z",
  "validBefore": "2026-10-22T12:00:00Z"
}
```

Listing 9: An add-key invocation.

```
{
  "id": 42,
  "trusted": true
}
```

Listing 10: An add-key response.

2.6 DELETE /certificate-authority/mgmt/keys/*{id}*

Operation: delete-key
Input: String
Output: RemovalResult

Deletes a trusted key by identifier (`{id}`).
Path parameter: *id*.
A successful call returns `200 OK` with the output in the body.

```
DELETE /certificate-authority/mgmt/keys/<id> HTTP/1.1
Host: certificate-authority:8448
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 11: An delete-key invocation.

```
{
  "success": true
}
```

Listing 12: An delete-key response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.8, which are used to represent things like hashes, identifiers and texts.

3.1 struct CertificateRecord

Object Field	Value Type	Description
"id"	Number	Record identifier.
"serial_number"	String	Serial number.
"common_name"	String	Subject common name.
"created_by"	String	Requesting system (`revoked-by:<caller>` for imported revocations).
"valid_from"	String	Start of validity.
"valid_until"	String	End of validity.
"revoked_at"	String	Revocation time, null if not revoked.
"certificate_pem"	String	The certificate, PEM.

3.2 struct RemovalResult

Object Field	Value Type	Description
"success"	Boolean	Whether a record was changed; 404 when false.

3.3 struct RevocationRequest

Object Field	Value Type	Description
"certificate"	String	Certificate to revoke, PEM or Base64 DER; must be issued by this cloud CA.

3.4 struct RevocationResult

Object Field	Value Type	Description
"id"	Number	Register record of the certificate.
"serialNumber"	String	Serial number.
"revoked"	Boolean	Always true.
"changed"	Boolean	False if it was already revoked.

3.5 struct TrustedKeyRecord

Object Field	Value Type	Description
"id"	Number	Record identifier.
"public_key_pem"	String	The key, PEM.
"description"	String	Description.
"valid_after"	String	Start of validity.
"valid_before"	String	End of validity.
"created_at"	String	Creation time.

3.6 struct TrustedKeyRequest

Object Field	Value Type	Description
"publicKey"	String	PEM SubjectPublicKeyInfo (alias `publicKeyPem`).
"description"	String	Description.
"validAfter"	DateTime	Start of validity; default now.
"validBefore"	DateTime	End of validity; default now plus `CERT_VALIDITY_DAYS`.

3.7 struct TrustedKeyResult

Object Field	Value Type	Description
"id"	Number	Record identifier.
"trusted"	Boolean	Always true.

3.8 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “certificateAuthorityManagement – Service Description (CertificateAuthority),” Eclipse Arrowhead, SD 5.2.2, 2026, certificateAuthority-v5.2.2-certificateAuthorityManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged; missing primitive definitions added.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	